

沈阳航空航天大学

《模式识别》实验报告

题目：感知机算法

院（系）： 人工智能学院

专 业： 物联网工程

班 级： 物联网2202

学 号： 223428010210

姓 名： 陈梓欣

指导教师： 刘翠微

1 任务描述

- A. 实验目的：通过编程和上机实验加深对监督学习的理解和认识，编程实现感知机分类算法，并在给定的数据集上测试算法的性能。
- B. 实验环境：PC 机，window 操作系统，python（包括 numpy、matplotlib、os 库）。
- C. 实验要求：
 - (1) 掌握讲授的模式分类算法
 - (2) 实现感知机分类算法，并在给定的数据集上测试算法性能。

2 实验内容

(1) 数据集的载入与划分

在本实验中，首先从文件 datasetTo.txt 中载入数据集，共包含 600 行样本。每行由三列组成：前两列为样本的二维坐标，第三列为类别标签（+1 表示正类样本，-1 表示负类样本）。

将前 200 行数据作为训练数据集（traindata），其余 400 行作为测试数据集（testdata）。在训练阶段可使用类别标签进行模型训练与验证。

通过 `numpy.loadtxt()` 函数读取数据，并使用 `matplotlib.pyplot.scatter()` 对样本的二维分布进行可视化，直观展示不同类别样本的分布特征。

(2) 感知机算法的实现

感知机（Perceptron）是线性可分数据的经典二分类算法，其基本思想是通过迭代调整权重向量 w 和偏置 b ，使所有样本满足分类条件 $y_i(wx_i + b) > 0$ 。

本实验要求手动实现感知机算法，不得调用现成的库函数。算法步骤如下：

- i. 初始化权重 $w = [0, 0]$ ，偏置 $b = 0$ ；

- ii. 设置学习率 η (取值范围 0.1, 0.2, ..., 0.9), 最大迭代次数 1000;
- iii. 对训练集样本逐个检查分类结果:
 - 若 $y_i(wx_i + b) \leq 0$, 则模型分类错误, 需要更新参数: $w = w + \eta y_i x_i$,
 $b = b + \eta y_i$
 - 若分类正确, 则跳过;
- iv. 若一轮遍历中无样本被误分, 则算法收敛, 停止训练。

训练完成后, 得到感知机模型参数 w 和 b 。

(3) 模型训练与分类平面可视化

在训练过程中, 分别设置不同的学习率 $\eta = 0.1, 0.3, 0.5, 0.7, 0.9$ 进行实验。

对于每个学习率, 记录模型最终的参数 w 和 b 。

使用二维平面图绘制分类结果:

- 训练集样本点以红、蓝两色区分正负类;
- 学习到的分类平面(直线)通过方程 $w_1 x_1 + w_2 x_2 + b = 0$ 表示, 在图中以虚线绘制。

通过对比不同 η 下的直线, 可以观察学习率变化对分类边界的影响。

(4) 模型选择与结果评估

使用训练得到的参数 w 和 b , 对测试集样本进行分类预测:

$$\hat{y} = \text{sign}(wx + b)$$

将预测结果与真实标签 testlabel 进行比较, 计算分类正确率:

$$\text{Accuracy} = \frac{\text{预测正确的样本数}}{\text{总测试样本数}}$$

使用散点图显示测试结果:

- 红色表示预测为正类;
- 蓝色表示预测为负类;
- 黑色虚线为分类平面。

通过比较测试集准确率和可视化效果, 评估感知机模型在不同学习率下的泛化能力。

3 实验结果和分析

A. 实验结果

(1) 载入数据集中的数据

载入数据集中的数据, 并将训练集中的所有样本显示在二维图像中, 红色表示正样本, 蓝色表示负样本。原始数据加载效果如图 1 所示。

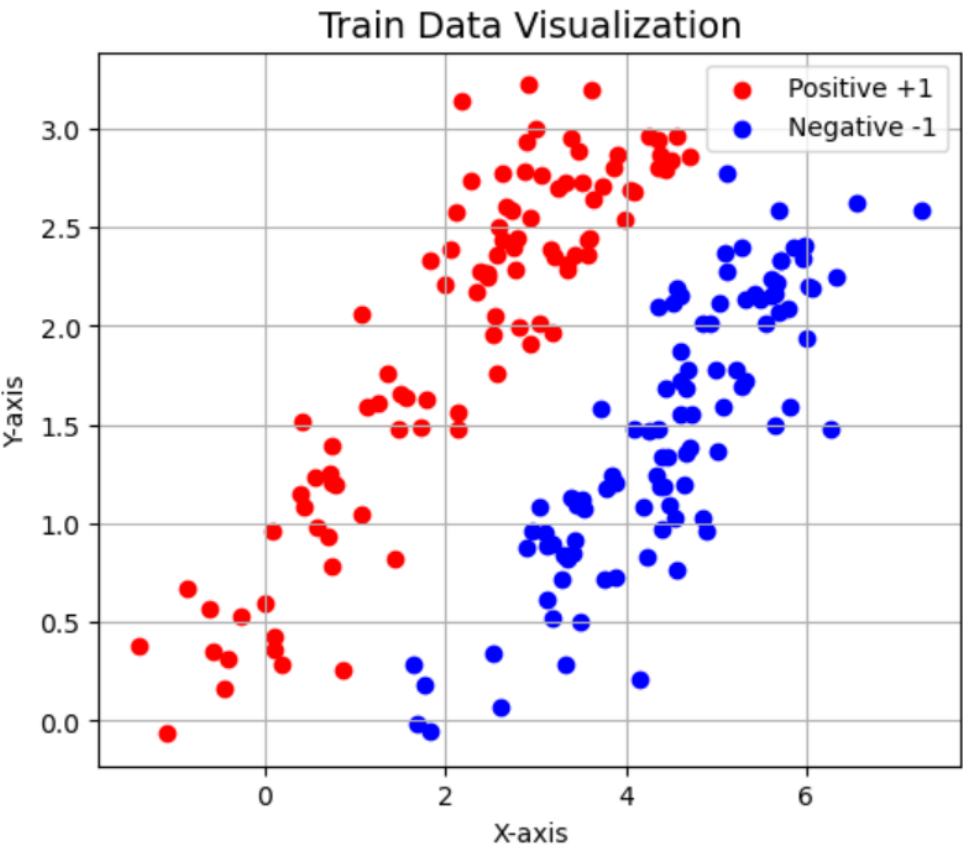


图 1 原原始数据加载效果图

(2) 编程实现感知机算法

采用手动编程方式实现感知机分类算法, 设定不同的学习率 $\eta = 0.1, 0.3, 0.5, 0.7, 0.9$, 对训练数据进行迭代学习, 逐步调整权重 w 和偏置 b 。训练过程中输出参数更新结果与误分类数。

实验结果表明，模型在若干次迭代后均能收敛。以 $\eta = 0.3$ 为例，最终得到的模型参数为：

- 权重 $w = [-2.95634309 \quad 3.89183859]$
- 偏置 $b = 3.300$
- 测试集分类准确率 = 98.50%

具体训练过程如图 2 和图 3 所示，不同学习率下感知机算法训练收敛曲线如图 4 所示。

```
--- 开始训练 ( $\eta = 0.3$ ) ---
初始参数:  $w = [0. \ 0.]$ ,  $b = 0$ 

第 1 轮, 第 1 个样本被误分类 → 更新后  $w = [0.22075055 \ 0.36181529]$ ,  $b = 0.300$ 
第 1 轮, 第 101 个样本被误分类 → 更新后  $w = [-1.96325784 \ -0.41352066]$ ,  $b = 0.000$ 
第 1 轮结束, 误分类样本数 = 2

第 2 轮, 第 1 个样本被误分类 → 更新后  $w = [-1.74250729 \ -0.05170537]$ ,  $b = 0.300$ 
第 2 轮, 第 2 个样本被误分类 → 更新后  $w = [-1.52636297 \ 0.32499279]$ ,  $b = 0.600$ 
第 2 轮, 第 3 个样本被误分类 → 更新后  $w = [-1.18884989 \ 0.80221098]$ ,  $b = 0.900$ 
第 2 轮, 第 20 个样本被误分类 → 更新后  $w = [-0.75901498 \ 1.04874912]$ ,  $b = 1.200$ 
第 2 轮, 第 111 个样本被误分类 → 更新后  $w = [-2.06936871 \ 0.42023436]$ ,  $b = 0.900$ 
第 2 轮结束, 误分类样本数 = 5

第 3 轮, 第 1 个样本被误分类 → 更新后  $w = [-1.84861816 \ 0.78204965]$ ,  $b = 1.200$ 
第 3 轮, 第 12 个样本被误分类 → 更新后  $w = [-1.39891226 \ 1.27814215]$ ,  $b = 1.500$ 
第 3 轮, 第 32 个样本被误分类 → 更新后  $w = [-0.12175054 \ 2.16536438]$ ,  $b = 1.800$ 
第 3 轮, 第 101 个样本被误分类 → 更新后  $w = [-2.30575893 \ 1.39002843]$ ,  $b = 1.500$ 
第 3 轮结束, 误分类样本数 = 4

第 4 轮, 第 20 个样本被误分类 → 更新后  $w = [-1.87592402 \ 1.63656657]$ ,  $b = 1.800$ 
第 4 轮, 第 32 个样本被误分类 → 更新后  $w = [-0.5987623 \ 2.5237888]$ ,  $b = 2.100$ 
第 4 轮, 第 101 个样本被误分类 → 更新后  $w = [-2.7827707 \ 1.74845285]$ ,  $b = 1.800$ 
第 4 轮结束, 误分类样本数 = 3

第 5 轮, 第 20 个样本被误分类 → 更新后  $w = [-2.35293579 \ 1.99499099]$ ,  $b = 2.100$ 
第 5 轮, 第 32 个样本被误分类 → 更新后  $w = [-1.07577407 \ 2.88221322]$ ,  $b = 2.400$ 
第 5 轮, 第 101 个样本被误分类 → 更新后  $w = [-3.25978246 \ 2.10687727]$ ,  $b = 2.100$ 
第 5 轮结束, 误分类样本数 = 3

第 6 轮, 第 20 个样本被误分类 → 更新后  $w = [-2.82994755 \ 2.3534154]$ ,  $b = 2.400$ 
第 6 轮, 第 21 个样本被误分类 → 更新后  $w = [-1.99732889 \ 3.04029414]$ ,  $b = 2.700$ 
第 6 轮, 第 111 个样本被误分类 → 更新后  $w = [-3.30768262 \ 2.41177938]$ ,  $b = 2.400$ 
第 6 轮结束, 误分类样本数 = 3

第 7 轮, 第 20 个样本被误分类 → 更新后  $w = [-2.87784771 \ 2.65831751]$ ,  $b = 2.700$ 
第 7 轮, 第 32 个样本被误分类 → 更新后  $w = [-1.60068599 \ 3.54553975]$ ,  $b = 3.000$ 
第 7 轮, 第 101 个样本被误分类 → 更新后  $w = [-3.78469439 \ 2.77020379]$ ,  $b = 2.700$ 
第 7 轮结束, 误分类样本数 = 3
```

图 2 $\eta = 0.3$ 具体训练过程 (1)

第 8 轮, 第 20 个样本被误分类 → 更新后 $w = [-3.35485947 \quad 3.01674193]$, $b = 3.000$
第 8 轮, 第 32 个样本被误分类 → 更新后 $w = [-2.07769776 \quad 3.90396416]$, $b = 3.300$
第 8 轮, 第 103 个样本被误分类 → 更新后 $w = [-3.78896175 \quad 3.20495985]$, $b = 3.000$
第 8 轮结束, 误分类样本数 = 3

第 9 轮, 第 21 个样本被误分类 → 更新后 $w = [-2.95634309 \quad 3.89183859]$, $b = 3.300$
第 9 轮结束, 误分类样本数 = 1

模型在第 10 轮迭代后收敛!
最终参数: $w = [-2.95634309 \quad 3.89183859]$, $b = 3.300$

图 3 $\eta = 0.3$ 具体训练过程 (2)

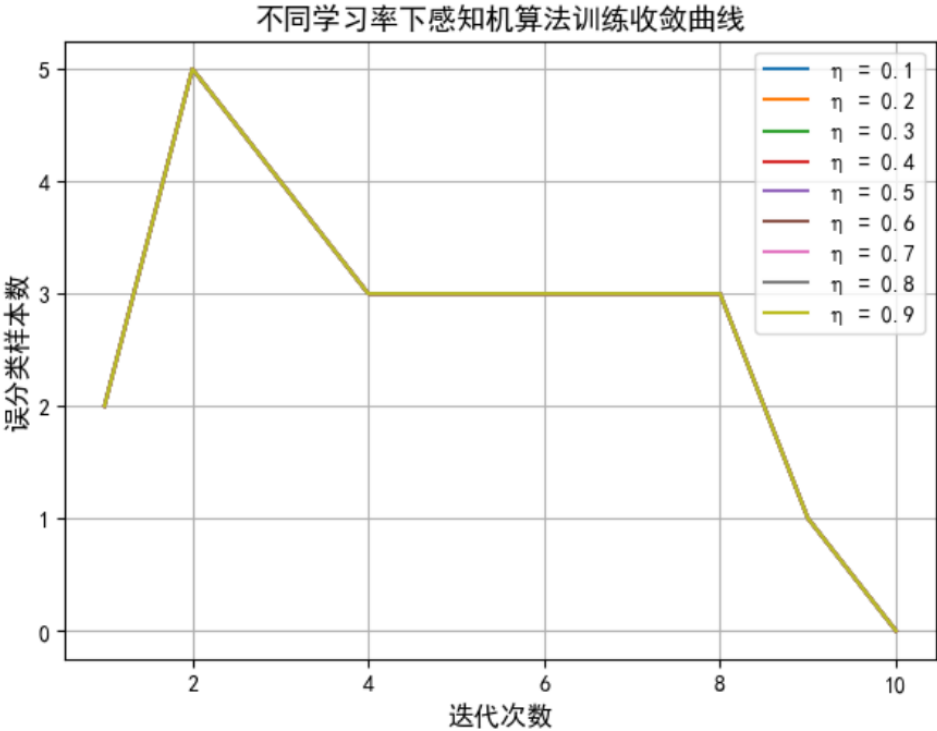


图 4 不同学习率下感知机算法训练收敛曲线 (已重合)

(3) 测试集分类与结果分析

使用训练得到的参数 w 和 b 对测试集样本进行预测分类。结果显示, 模型能较好地区分正负样本类别。分类平面 (直线) 方程为: $w_1x_1 + w_2x_2 + b = 0$ 。

在二维图中绘制该分界线后可见, 大部分样本被正确划分到对应区域。不同学习率下的分类结果如图 3 所示, 其中 $\eta = 0.3$ 和 $\eta = 0.5$ 时效果最佳, 分类边界稳定且准确率较高。

不同学习率 η 下测试数据分类结果如图 5-图所示。

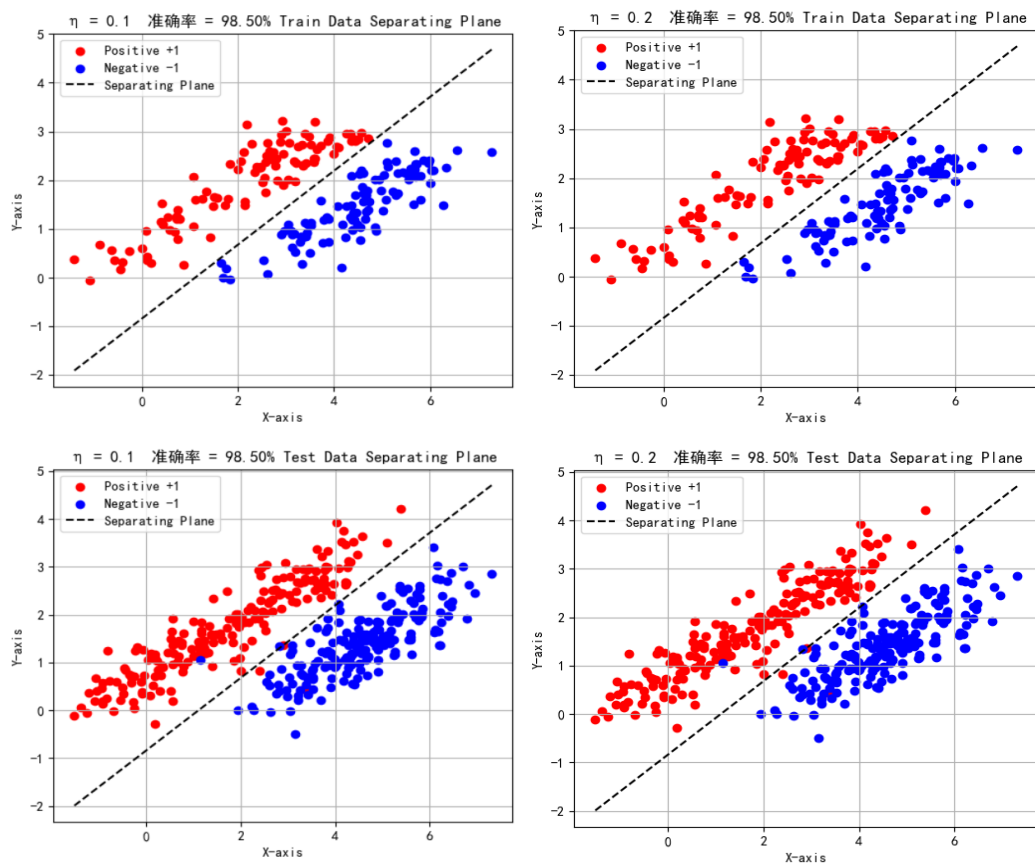


图 5 学习率 $\eta=0.1$ 和 0.2 下测试数据分类结果

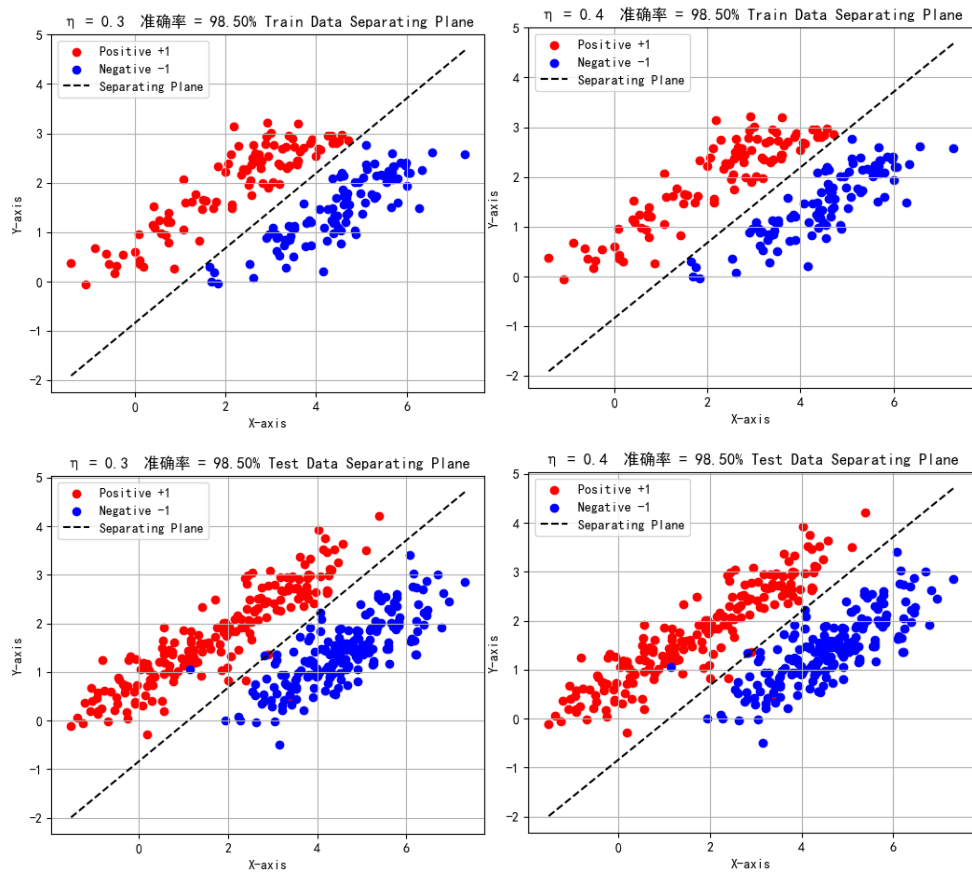


图 6 学习率 $\eta=0.3$ 和 0.4 下测试数据分类结果

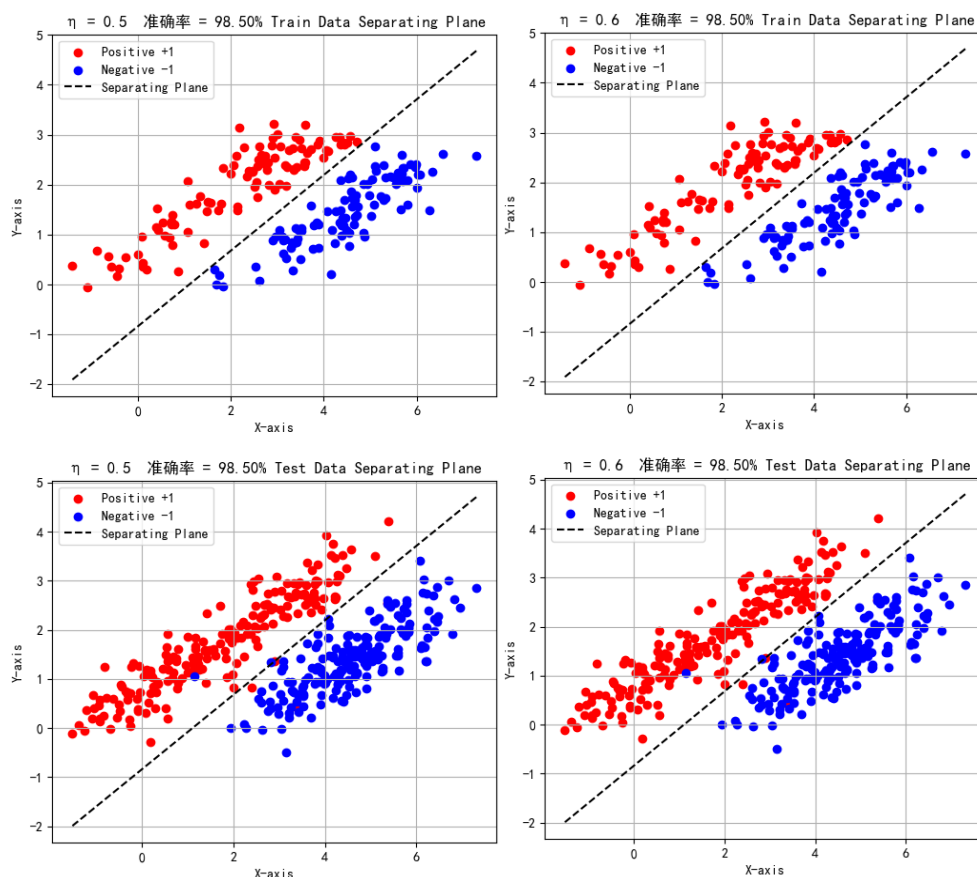


图 7 学习率 $\eta=0.5$ 和 0.6 下测试数据分类结果

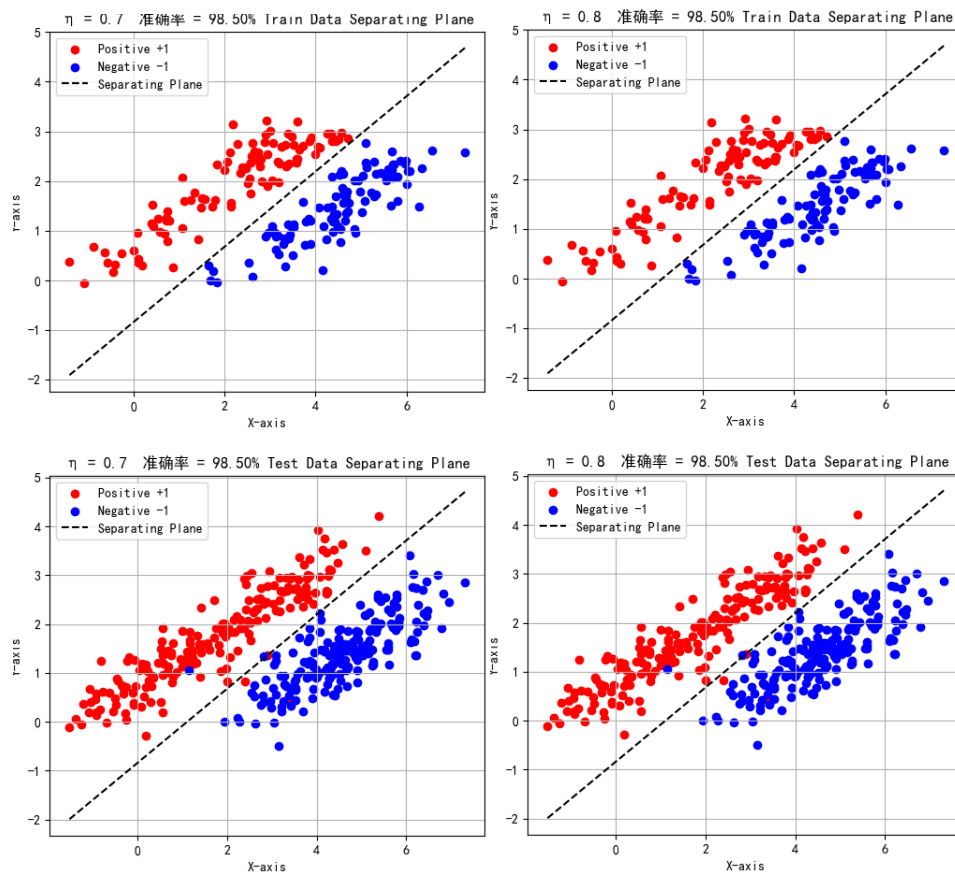


图 8 学习率 $\eta=0.7$ 和 0.8 下测试数据分类结果

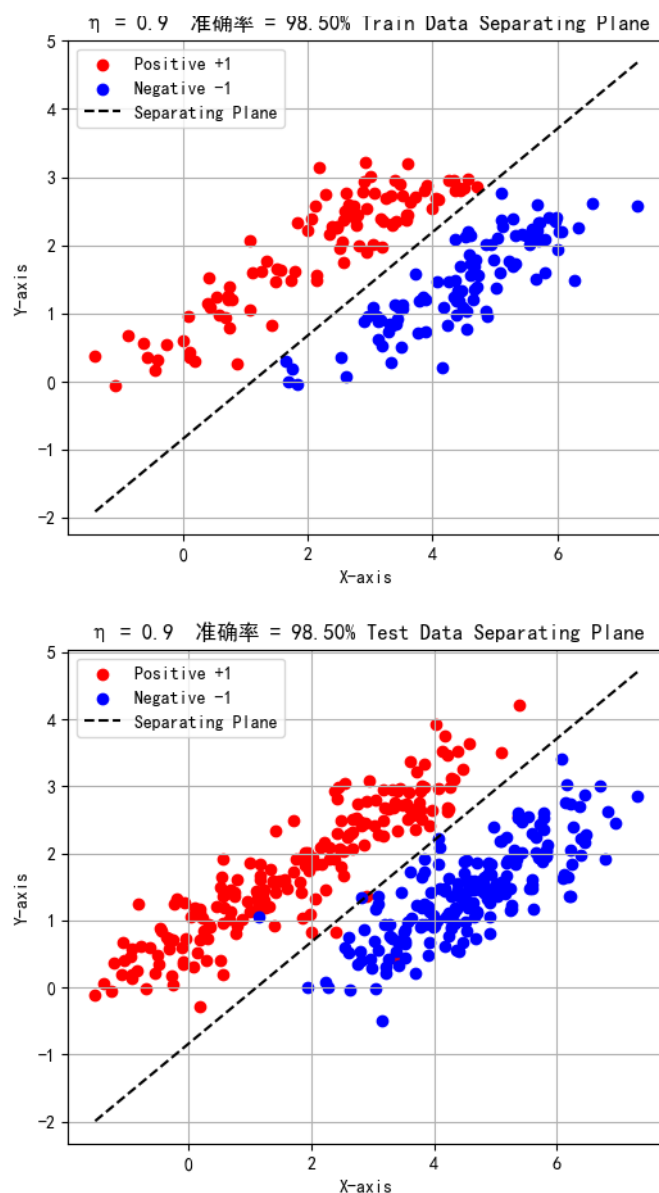


图 9 学习率 $\eta=0.9$ 下测试数据分类结果

B. 实验分析

(1) 分析在不同的学习率 η 下，感知机模型的分类平面有何不同

在感知机算法中，学习率 η 是影响模型训练过程和分类平面形态的重要参数。

实验结果表明，当学习率 η 较小时（如 0.1），模型参数更新幅度较小，权重调整趋于平稳，分类平面收敛速度慢但较为稳定，最终得到的分界线较为平滑、接近最优线性边界；

当 η 适中（如 0.3 ~ 0.5）时，模型在合理的迭代次数内能快速收敛，分类效果较好，训练误差较低；而当 η 过大（如 0.7 ~ 0.9）时，权重更新步长过大，模型容易在样本两侧来回震荡，导致分类平面不稳定甚至无法收敛，出现边界摆动或误分类现象。

因此，学习率需要在过小和过大之间权衡，以兼顾训练速度和分类精度。综合来看，中等大小的学习率通常能获得更理想的分类平面和更高的模型性能。

(2) 实验结论

本实验旨在通过编程实现感知机算法，加深对监督学习及线性分类思想的理解。实验环境为 Python（含 numpy、matplotlib 库），操作系统为 Windows。数据集共 600 个样本点，前 200 行作为训练集，后 400 行为测试集。样本为二维坐标点，标签取值为 1 或 -1。

实验首先利用 `numpy.loadtxt` 函数加载数据，并绘制训练数据的分布图，红色代表正类，蓝色代表负类。随后，手动实现感知机训练算法，不调用库函数。算法通过不断调整权重向量 w 与偏置 b ，使所有训练样本满足分类条件 $y_i(wx_i + b) > 0$ 。

在训练过程中分别设置不同的学习率 $\eta = 0.1, 0.3, 0.5, 0.7, 0.9$ ，对模型进行训练并绘制分类平面。

测试阶段，利用学习到的参数对测试数据进行分类，计算准确率并绘制分类结果图。实验结果表明，学习率 η 对模型的收敛速度与分类平面稳定性影响显著： η 过小时模型收敛慢， η 过大时容易震荡不稳。综合分析，适中的学习率（如 0.3 ~ 0.5）能在较短时间内收敛并得到稳定、合理的分类平面。本实验有效验证了感知机算法的可行性及学习率参数的重要作用。

4 附录

数据载入与划分

datasetTo.txt 共 600 行：前 200 行为训练集，后 400 行为测试集

```
data = np.loadtxt("data/datasetTo.txt")
```

```
X_train = data[:200, :2]    # 训练数据的二维特征
```

```
y_train = data[:200, 2]    # 训练标签 (1 或 -1)
```

```
X_test = data[200:, :2]    # 测试数据
```

```
y_test = data[200:, 2]    # 测试标签
```

设置中文字体与负号正常显示

```
# plt.rcParams['font.sans-serif'] = ['SimHei']    # 显示中文
```

```
# plt.rcParams['axes.unicode_minus'] = False      # 正常显示负号
```

显示训练数据的分布

```
plt.figure(figsize=(6, 5))
```

```
plt.title("Train Data Visualization", fontsize=14)
```

```
plt.scatter(X_train[y_train == 1][:, 0], X_train[y_train == 1][:, 1], c='r', label='Positive +1')
```

```
plt.scatter(X_train[y_train == -1][:, 0], X_train[y_train == -1][:, 1], c='b', label='Negative -1')
```

```
plt.legend()
```

```
plt.xlabel("X-axis")
```

```
plt.ylabel("Y-axis")
```

```
plt.grid(True)
```

```
plt.show()
```

感知机训练算法实现

```
def perceptron_train(X, y, eta=0.1, max_iter=1000):
```

```
    n_samples, n_features = X.shape
```

```
    w = np.zeros(n_features)
```

```
b = 0
```

```
for epoch in range(max_iter):
    error_count = 0
    for i in range(n_samples):
        # 感知机判别函数
        if y[i] * (np.dot(w, X[i]) + b) <= 0:
            # 更新参数
            w += eta * y[i] * X[i]
            b += eta * y[i]
            error_count += 1
    if error_count == 0:
        break
```

```
return w, b
```

```
# 测试与可视化
```

```
def plot_decision_boundary(X, y, w, b, title):
    plt.figure(figsize=(6, 5))
    plt.scatter(X[y == 1][:, 0], X[y == 1][:, 1], c='r', label='Positive +1')
    plt.scatter(X[y == -1][:, 0], X[y == -1][:, 1], c='b', label='Negative -1')

    # 决策平面（直线）： $w_1 * x_1 + w_2 * x_2 + b = 0 \rightarrow x_2 = -(w_1 * x_1 + b) / w_2$ 

    x1 = np.linspace(min(X[:,0]), max(X[:,0]), 100)
    x2 = -(w[0]*x1 + b)/w[1]
    plt.plot(x1, x2, 'k--', label='Separating Plane')

    plt.title(title)
    plt.legend()
    plt.xlabel("X-axis")
    plt.ylabel("Y-axis")
    plt.grid(True)
    plt.show()
```

```
# 在不同学习率下训练与测试
```

```
etas = [0.1, 0.3, 0.5, 0.7, 0.9]
```

```
for eta in etas:
```

```
    w, b = perceptron_train(X_train, y_train, eta)
```

```
    # 对测试集进行分类预测
```

```
    y_pred = np.sign(np.dot(X_test, w) + b)
```

```
accuracy = np.mean(y_pred == y_test)
```

```
print(f"学习率  $\eta$  = {eta:.1f} 时:")
```

```
print(f" 训练得到的权重  $w$  = {w}")
```

```
print(f" 偏置  $b$  = {b:.3f}")
```

```
print(f" 测试集分类准确率 = {accuracy*100:.2f}%\n")
```

```
plot_decision_boundary(X_train, y_train, w, b, title=f" $\eta$  = {eta:.1f} Train  
Data Separating Plane")
```

```
plot_decision_boundary(X_test, y_test, w, b, title=f" $\eta$  = {eta:.1f} Test Data  
Separating Plane")
```